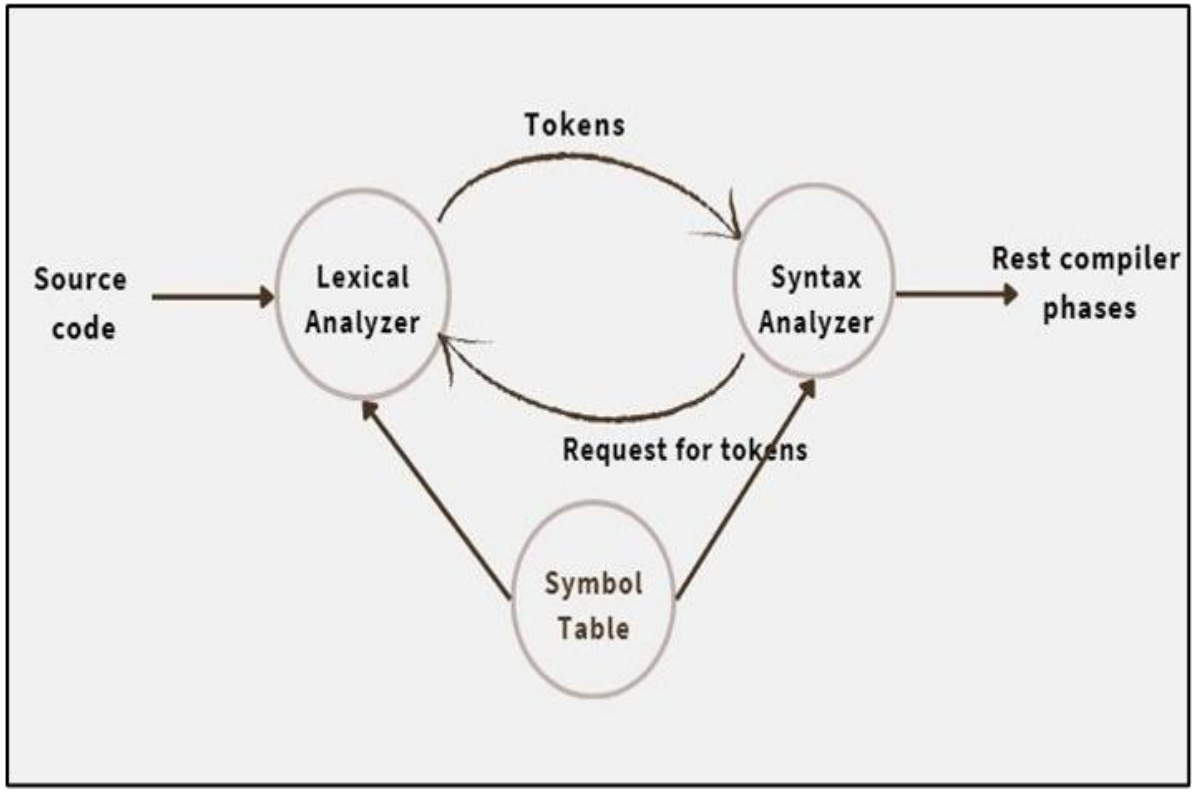
Unit 2

SYNTAX ANALYSIS

- Its role
- Basic parsing techniques:
  - Problem of Left Recursion
  - Left Factoring
  - Ambiguous Grammar
  - Top-down parsing
  - Bottom-up parsing
  - LR parsing



Syntax Analysis

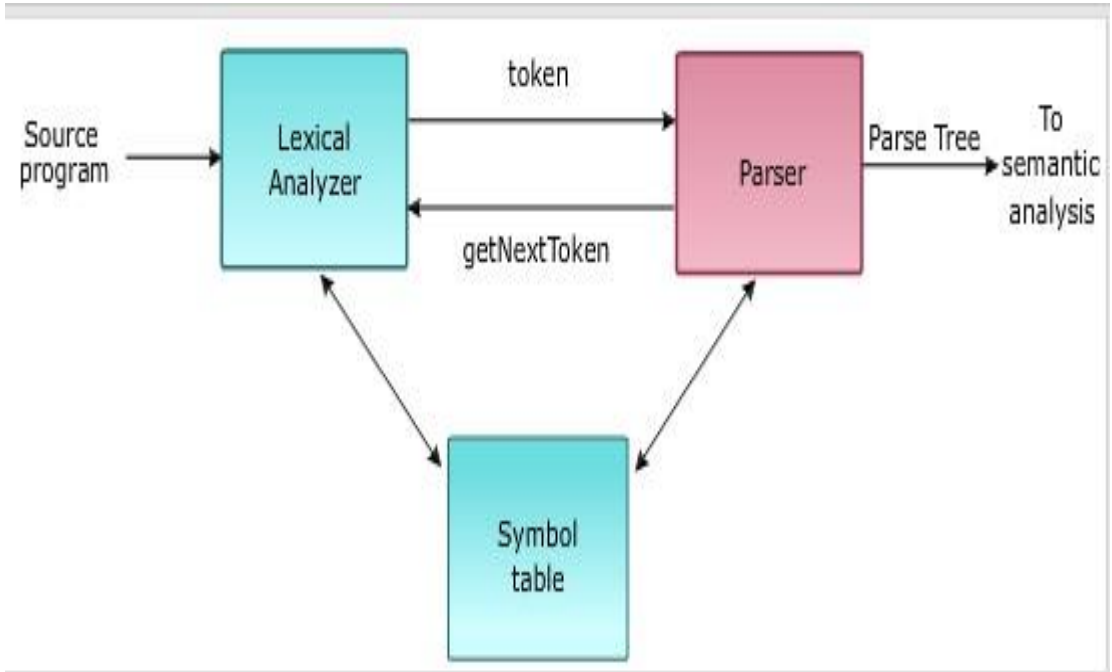
- Syntax Analysis is the second phase of a compiler.
- It receives tokens from the lexical analyzer.
- It checks whether the sequence of tokens follows the grammar rules of the programming language.
- If the token sequence is grammatically correct, it generates a parse tree.
- If the token sequence is grammatically incorrect, it reports a syntax error.
- Example

  Input: a = b + c

  Tokens received from the lexical analyzer: ID = ID + ID

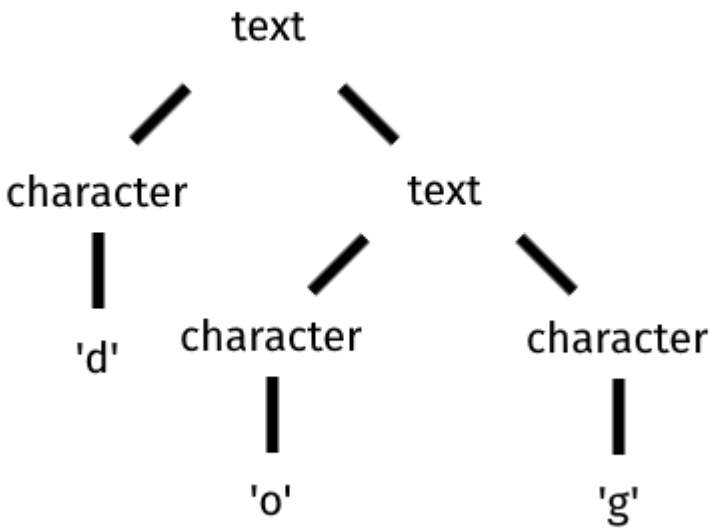
Result:

- If the tokens follow the grammar, the parser **accepts** the input and generates a **parse tree**.
- Otherwise, it **reports a syntax error**.



Interaction Between Lexical Analyzer and Parser

Parse Tree



Role of Syntax Analysis

- **Checks grammar**: Ensures that the program follows the grammar rules of the programming language.
- **Builds a Parse Tree**: Creates a parse tree to show the grammatical structure of the program.
- **Detects syntax errors**: Finds errors such as missing semicolons, missing brackets, or incorrect statement structure.
- **Reports errors**: Displays clear error messages to help the programmer correct the mistakes.
- **Passes the Parse Tree**: Sends the parse tree to the **Semantic Analysis** phase for further checking.

Parser

- A **parser** is a part of the compiler that receives **tokens** from the lexical analyzer and checks whether they follow the **grammar rules** of the programming language.
- If the sequence of tokens is correct, the parser creates a **parse tree**.
- If the syntax is incorrect, it reports **syntax errors**.

Working of a Parser

- **Input:** Receives tokens generated by the lexical analyzer.
- **Process:** Checks whether the tokens follow the grammar rules of the language.
- **Output:** Generates a parse tree for valid syntax or reports syntax errors for invalid syntax.

Tasks of a Parser in Compiler Design

- **Checks Syntax:** Verifies whether the program follows the grammar rules of the programming language.
- **Detects Syntax Errors:** Finds errors in the source code.
- **Locates Errors:** Identifies the position where the error occurs.
- **Reports Errors:** Displays clear and meaningful error messages.
- **Performs Error Recovery:** Continues parsing after an error to find more errors.
- **Accepts Correct Programs:** Successfully parses programs with correct syntax.
- **Rejects Incorrect Programs:** Reports syntax errors for programs that do not follow the grammar rules.

Basic Parsing Techniques

Parsing is the process of analyzing a sequence of tokens to check whether they follow the grammar rules of a programming language.
It determines the structure of the program and builds a **parse tree**.
There are two main approaches to parsing:

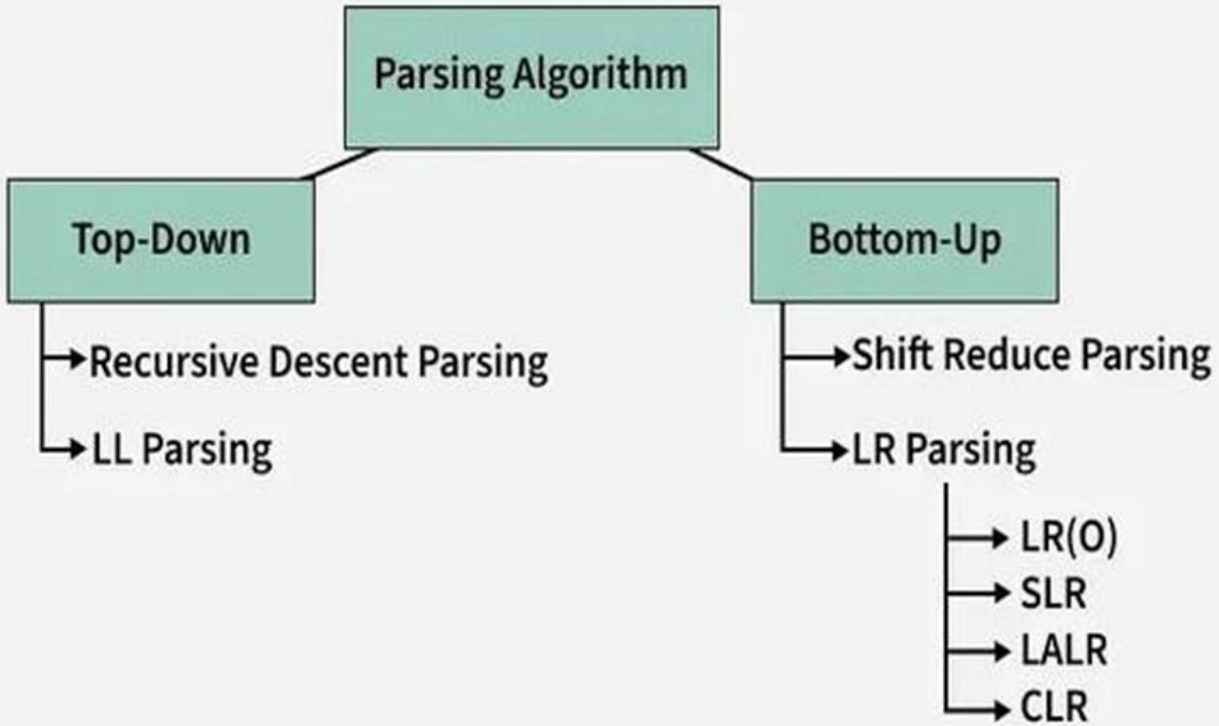
1. Top-Down Parsing

- Top-down parsing builds the parse tree from the root to the leaves by starting from the start symbol and applying grammar rules.
- Idea:
  - Starts from the start symbol of the grammar.
  - Breaks the grammar into smaller parts to match the input tokens.
- Methods:
  - Recursive Descent Parsing
  - Predictive (LL) Parsing

2. Bottom-Up Parsing

- Bottom-up parsing builds the parse tree from the leaves to the root by starting with input tokens and combining them to reach the start symbol.
- Idea:
  - Starts from the input tokens.
  - Combines tokens step by step until the complete grammar structure is formed.
- Methods:
  - Shift-Reduce Parsing
  - LR Parsing

Parsing Algorithm Used in Syntax Analysis



Left Recursion in a Grammar

- A grammar is said to have left recursion if a non-terminal calls itself as the leftmost symbol in one of its production rules.
- In other words, a production is left-recursive if it has the form:

$$A \rightarrow A\alpha$$

- where:

  A is a non-terminal.

  α is any sequence of terminals and/or non-terminals.

- Left recursion causes a top-down parser (such as recursive descent or predictive parsing) to enter an infinite recursion (loop).
- Therefore, left recursion must be eliminated before using predictive (LL) parsing.

EXAMPLE:

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow id$$

Here, the production:

$$E \rightarrow E + T$$

is **left-recursive** because **E** appears as the leftmost symbol on the right-hand side.

Problems with Left Recursion

- It causes infinite recursion (or an infinite loop) in top-down parsers such as such as recursive descent and LL(1) parsers.

Major issues with left recursion

i. Infinite Recursion

- Left-recursive production causes the parser to repeatedly call the same parsing function.
- This results in an infinite loop or stack overflow.

Example**:**

$$E \rightarrow E + T \mid T$$

Here E calls itself before reading any input.

```
parseE()
  → parseE()
    → parseE()
      → ...
```

ii. Stack Overflow

- Since the parser keeps making recursive calls, the program's call stack grows until it exceeds its limit, causing a stack overflow.

iii. Parser Cannot Make Progress

- The parser repeatedly expands the same nonterminal without consuming any input symbols.
- As a result, it never reaches the terminal symbols needed for parsing.

iv. Not Suitable for LL(1) Parsing

- LL(1) parsers require grammars without left recursion.
- Left-recursive grammars cannot be directly used to construct LL(1) parsing tables.

v. Increases Parsing Complexity

- Left-recursive grammars complicate parser implementation and must be transformed before using top-down parsing techniques.

Solution:

- Left recursion is eliminated by rewriting the grammar into an equivalent grammar that is not left recursive.
- Use left factoring only when two or more productions have a common prefix. Left factoring is not the technique for removing left recursion.

Types of Left Recursion

1. Direct Left Recursion

- A non-terminal immediately calls itself.

$$A \rightarrow A\alpha \mid \beta$$

where:

A is non terminal

α is any sequence of grammar symbols

β is a production that does not begin with A

- Example: $E \rightarrow E + T \mid T$

$T \rightarrow id$

2. Indirect Left Recursion

- A non-terminal calls itself through one or more other non-terminals.
- Example:

$$A \rightarrow B\alpha$$

$$B \rightarrow A\beta \mid c$$

Here

$$A \Rightarrow B\alpha \Rightarrow A\beta\alpha$$

- So, **A** indirectly calls itself, making the grammar indirectly left-recursive.

How to eliminate left recursion

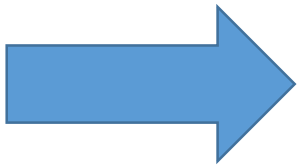
We can eliminate left recursion by replacing a pair of production

$A \rightarrow A\,\alpha \,|\beta$ with:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' | \epsilon$

Example 1: S-> Sb|a



S->aS’

S’->bS’| ϵ

Example 2:

A → ABd / Aa / a

B → Be / b

A → aA’

A’ → BdA’ / aA’ / ϵ

B → bB’

B’ → eB’ / ϵ

Example 3:

$A \rightarrow B\ a|c$

$B \rightarrow A\ b|d$

Step1 : Look for indirect recursion if any

Substitute A into B, we get

$B \rightarrow (Ba|c)\ b \mid d$

$B \rightarrow Bab \mid cb \mid d$

Now it becomes **direct left recursion**.

Step 2: Eliminate Direct Left Recursion

- Introduce any new non terminal say B'

- Grammar becomes:

$A \rightarrow B\ a|c$

$B \rightarrow cbB' \mid dB'$

$B' \rightarrow abB' \mid \epsilon$

This grammar has no let recursion now

Remove left Recursion of given grammar

1. $E \rightarrow E+T|T$

   $T \rightarrow T*F|F$

   $F \rightarrow (E)|id$

   $E \rightarrow TE'$

   $E' \rightarrow +TE'|\ \varepsilon$

   $T \rightarrow FT'$

   $T \rightarrow *\ FT'|\varepsilon$

   $F \rightarrow (E)|\ id$

2. $S \rightarrow a|^|(T)$

   $T \rightarrow T,\ S|S$

   $S \rightarrow a|^|(T)$

   $T \rightarrow ST'$

   $T' \rightarrow ,ST'|\varepsilon$

3. [TU 2078]

   S→SB | Ca
   B→Bb | c
   C→aB | a

Left Factoring

- **Left factoring** is a grammar transformation technique used to remove the **common prefix** from two or more productions of the same nonterminal.
- It helps a **predictive (LL(1)) parser** decide which production to use by delaying the decision until enough input has been read.
- Left factoring is **not** used to remove left recursion. It is used only when productions share a common prefix.

Why is Left Factoring Needed?

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

Both productions start with the common prefix α.

A predictive parser cannot determine which production to choose after seeing α

Left factoring removes this ambiguity

Example:

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$

After Left Factoring:

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

where:
α is common prefix
β_1 and β_2 remaining parts of the productions

Ambiguous Grammar

- An ambiguous grammar is a grammar in which the same input string can be generated in more than one way.
- A grammar is ambiguous if at least one string in the language has more than one parse tree.
- This means the same input string has:
  - More than one parse tree, or
  - More than one leftmost derivation, or
  - More than one rightmost derivation.

Causes of Ambiguity

The two main causes are:

- Operator Precedence : Precedence determines which operator is evaluated first.
- Operator Associativity : determines the order of evaluation when operators have the same precedence.

Example:

Consider the grammar:

$$E \rightarrow E+E \mid E*E \mid id$$

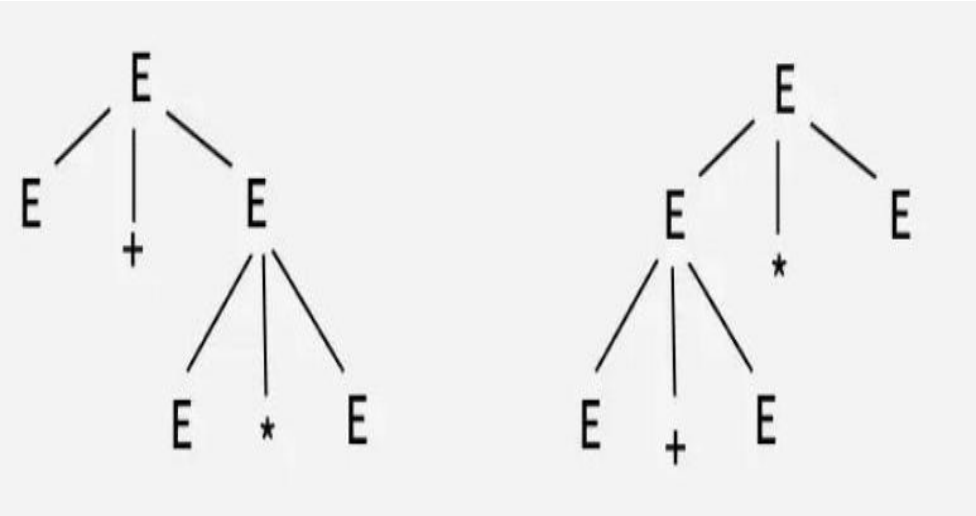
Let any string be: id + id ∗ id

This string can be parsed in two different ways.

Groups Addition first: (id+id)∗id

Groups Multiplication first: id+(id∗id)

Since the same string has **two different parse trees**, the grammar is **ambiguous**.



More than one parse tree

E → **E** + E
→ id + **E**
→ id + **E** x E
→ id + id x **E**
→ id + id x id

E → **E** x E
→ **E** + E x E
→ id + **E** x E
→ id + id x **E**
→ id + id x id

More than one leftmost derivation

Why is Ambiguity a Problem?

- Ambiguity is a problem because the same input can have more than one meaning.
- If a grammar produces multiple parse trees for the same statement, the compiler cannot decide which meaning is correct.
- This can lead to incorrect results during program execution.
- Therefore, a compiler needs a clear and unique parse tree to understand the program correctly and generate accurate machine code.

Example:

Any string : $2+3*4$

- This statement can be parsed in two ways
  - If parsed as $(2+3)*4$, the result is **20**.
  - If parsed as $2+(3*4)$, the result is **14**.
- Different parse trees produce different meanings.

Removing Ambiguity from a Grammar

- Removing ambiguity means rewriting a grammar so that every valid string has only one parse tree.
- The language does not change—only the grammar is rewritten to eliminate multiple possible parses.

Unambiguous Grammar

- A CFG is called unambiguous if every string generated by the grammar has only one parse tree.
- In other words, each string has:
  - Only one leftmost derivation.
  - Only one rightmost derivation.
- An unambiguous grammar gives a single, clear meaning to every input string, which helps the compiler generate correct results.

Example:

$E \rightarrow E + T \mid T$

$T \rightarrow id$

Input string: id + id

For this string, This grammar produces only one parse tree. [Verify this yourself]

Therefore, the grammar is unambiguous.

Top-Down Parsing

- **Top-down parsing** is a parsing technique in which the parser starts with the **start symbol** of the grammar and expands **non-terminals** until the input string is matched.
- The parser predicts which production rule to use at each step and compares the generated symbols with the input tokens.
- If the symbols match, parsing continues; otherwise, the parser reports an error or tries another rule (in parsers that support backtracking).
- In this Parsing
  - Parsing begins from the root of the parse tree.
  - The parse tree is built from top to bottom.
  - It always produces the leftmost derivation of the input string.

Top-Down Parsing : Procedure

Top-down parsing follows these main steps:

- Start with the start symbol

  Begin parsing from the grammar's start symbol (e.g., S).

- Apply production rules

  Replace the non-terminal with its production rules to match the input.

- Match input tokens

  Compare generated symbols with the input string step by step.

- Backtracking (if needed)

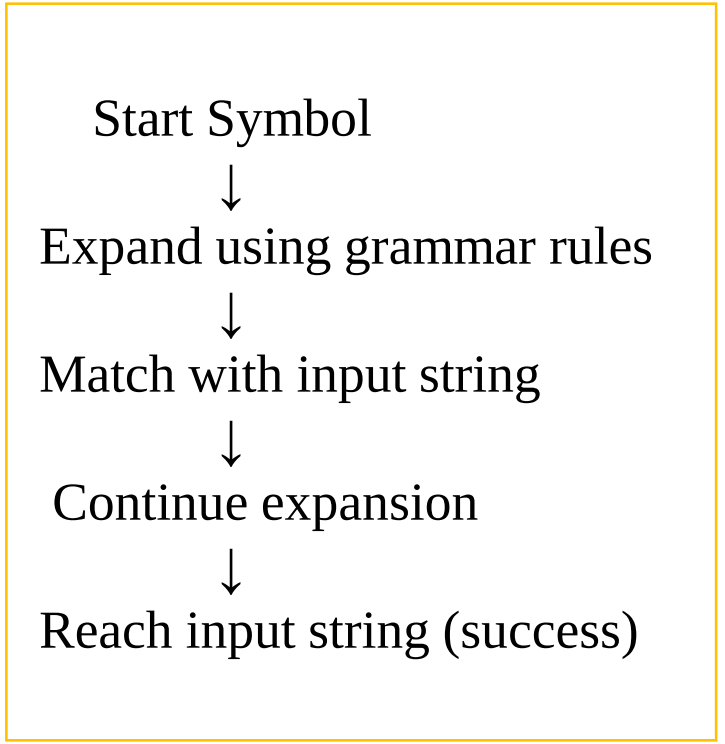
  If a chosen rule does not match, the parser may go back and try another rule

- Continue until completion

  Repeat expansion until:

  The input is fully matched → success

  No valid rule works → failure



Characteristics

- Starts with the start symbol.
- Builds the parse tree from top to bottom.
- Produces the leftmost derivation.
- Simpler to implement than bottom-up parsing.
- Cannot handle left-recursive grammars directly.
- Often requires left factoring to remove common prefixes.

Example: Top-Down Parsing

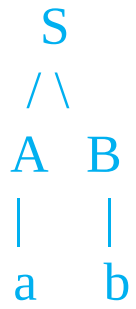
Grammar

- S → A B
- A → a
- B → b

Input String : ab

Step-by-step Top-Down Parsing

- Start S
- Apply rule S → A B
- Expand A → a a B
- Match a with input a
- Expand B → b a b
- Match b with input b



Result

- Input string **“ab” is successfully parsed**

Recursive Descent Parsing

- Recursive descent parsing is a top-down parsing technique.
- It uses one recursive function for each nonterminal in the grammar.
- Each function tries to match the input according to the production rules of that nonterminal.
- These functions call one another recursively to parse the input tokens based on the grammar.
- It is easy to understand and implement.
- **Limitations**:
- **Left recursion** causes infinite recursion, so the grammar must be modified to remove left recursion.
- **Backtracking** may occur if the parser chooses the wrong production and has to return to a previous point to try another one, making parsing slower.

Grammar:
S → aA
A → b
Input: ab

Functions:

```
S():
  match('a')
  A()

A():
  match('b')
```

Algorithm : Recursive Descent Parsing

- ✓Start with the start symbol.
- ✓Call the function for the start symbol.
- ✓If the current symbol is a terminal, compare it with the input.
- ✓If it matches, move to the next input symbol.
- ✓If the symbol is a nonterminal, call its function.
- ✓Continue until all input symbols are matched.
- ✓If all symbols match, accept; otherwise, reject.

```
void A() {
1)          Choose an A-production, A-> X1 X2 … Xk;

2)          for (i = 1 to k) {

3)                  if (Xi is a nonterminal)

4)                          call procedure Xi();

5)                  else if (Xi = current input symbol a)

6)                  advance the input to the next symbol;

7)                  else /* an error has occurred */;
```

Backtracking in Top-Down Parsing

- Backtracking is a technique used in top-down parsing when the selected production rule does not match the input.
- The parser returns to the point where it made the decision and tries another production rule for the same non-terminal.
- During backtracking, the parser:
  - Returns to the previous decision point.
  - Removes the failed parsing path (subtree).
  - Moves the input pointer back to the beginning of that parsing attempt.
  - Tries another production rule.
- Since the parser may scan the input more than once, backtracking takes more time and is less efficient.
- Therefore, backtracking parsers are rarely used in compiler design.

Example 1:

S → aA | aB

A → b

B → c

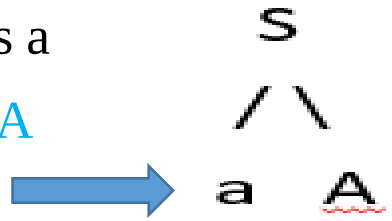
- Input String: ac

Solution:

Here First input symbol is a

Choosing grammar S-> aA

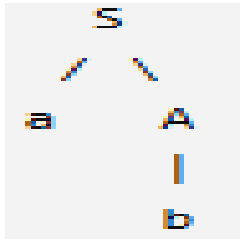
And Creating parse tree



The leftmost leaf a of the tree matches the first symbol of the input string a.

Thus, we will advance the input pointer to the next symbol of the input string c.

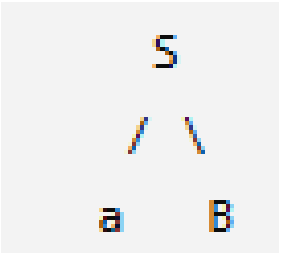
The next tree node is A. so expanding A → b, we get Parse tree as:



Here current input symbol c does not match with leaf node b, Thus the production S → aA fails to parse the string.

So **Backtrack** to previous decision point.

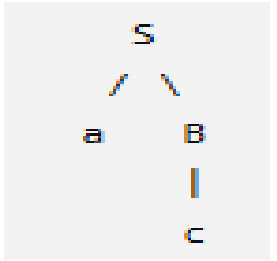
Now Choosing production S-> aB and expanding it:



The leftmost leaf a of the tree matches the first symbol of the input string a.

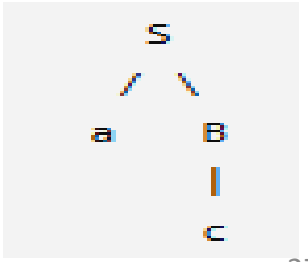
Now expand next node B with production B → c

Here current input symbol c matches with Leaf node c.



Since there are no more input symbols remaining, parsing completes.

And Final Parse tree is:



Parsing Process:

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

- Input String: ac

Input	Action / Rule Used	Output
ac	Start with S	—
ac	Use S → aA	aA
c	Match a	a
c	Use A → b	Expect b
c	b does not match c	Fail
ac	Backtrack	Return to S
ac	Use S → aB	aB
c	Match a	a
ε	Use B → c and match c	ac
ε	Parsing completed	Accepted

Class / Home Work

Recursive Descent Parsing Practices

Practice 1:

S → abA | abB

A → c

B → d

Input string: abd

Practice 2:

S → aAB | aC

A → b

B → d

C → bc

Input string : abc

Practice 3:

S → aS | bA

A → bA | c

Input String : aabc

Practice 4:

S -> aAd

A -> b | bc

Input String : abcd

Practice 5:

S → aA | b

A → cD | cE

D → d

E → e

Input String : ace

Practice 6:

S-> aBc | aaB
B-> bc | a
Input string= aaa

Non-Backtracking Parsing

- In non-backtracking parsing, the parser does not go back to try another rule after making a choice.
- It uses a predictive parsing table to choose the correct grammar rule.
- The parser looks at:
  - the top non-terminal on the stack, and
  - the next input symbol.
- Since it never goes back, it parses faster and is more efficient.
- The grammar should be left-factored and usually should not have left recursion.
- Major non-backtracking parsing methods are:
  - Predictive Parser and
  - LL(1) Parser

Predictive Parsing

- Predictive parsing is a simple form of recursive descent parsing.
- It is a top-down parsing technique used to check whether an input string follows the grammar of a language.
- Unlike recursive descent parsing, predictive parsing does not use backtracking.
- Once it selects a production rule, it does not return to try another rule.
- It uses a predictive parsing table to choose the correct production rule.
- The parser makes its decision based on:
  - the top non-terminal on the stack, and
  - the next input symbol, called the lookahead.
- Usually, the parser looks one input symbol ahead. This is known as LL(1) parsing.
- Using the parsing table, the parser predicts the correct production rule before continuing with the rest of the input.

Predictive Parsing : Steps

- Push the start symbol of the grammar onto the stack.
- Pop the top symbol from the stack.
- If it is a terminal:
  - Match it with the next input symbol.
  - If matched, move to the next input; otherwise, report an error.
- If it is a non-terminal:
  - Use the predictive parsing table and lookahead to choose the correct production.
  - Push the production's right-hand side onto the stack.
- Repeat until the stack and input are empty.
- Accept the input if both become empty; otherwise, reject it.

Advantages

- Easy to implement.
- No backtracking is required.
- Faster than recursive descent parsing with backtracking.
- Produces deterministic parsing.
- Detects syntax errors quickly.

Limitation

- It can parse only grammars that are left-factored and free from left recursion.

Components of a Predictive Top-Down Parser

1. Stack

- Stores the grammar symbols.
- Initially contains the **start symbol** on top of the $ end marker.

2. Input Buffer

- Stores the input string to be parsed.
- Ends with the $ symbol, which marks the end of the input.

3. Parsing Table

- Helps the parser choose the correct production rule.
- It is built using the **FIRST** and **FOLLOW** sets.

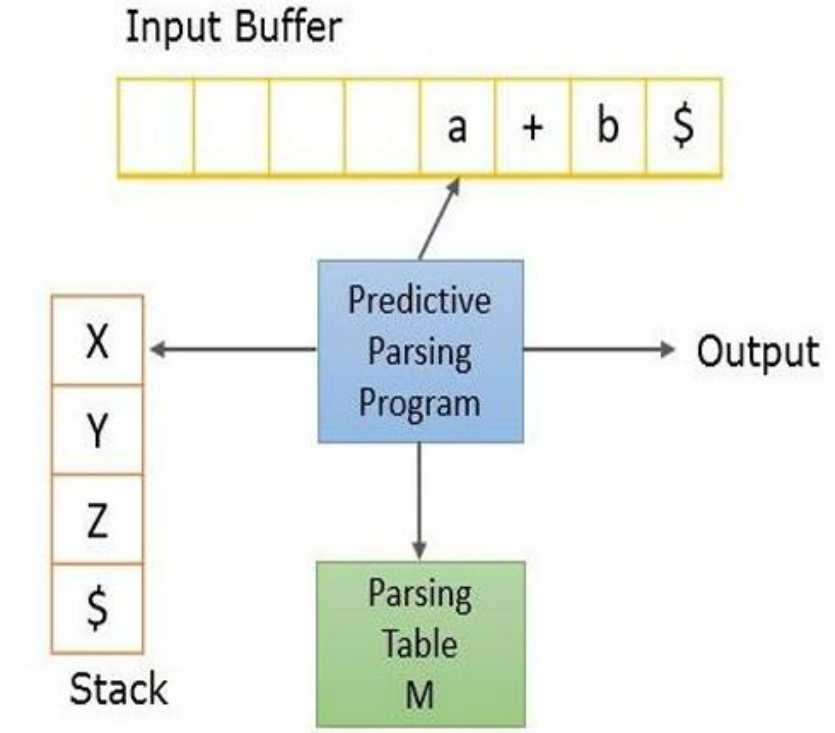


Table-Driven Predictive Recursive Parsing

Remember

- Both the stack and the input buffer contain the $ end marker.
- $ indicates the bottom of the stack and the end of the input string.
- At the beginning, the start symbol is placed on top of $ in the stack.

Example1:

Given grammar

S → aA
A → b

Input String : **ab$**

Initial Stack: S$

Parsing Process explanation:

- The parser starts with **S** on the stack.
- It uses the parsing table to choose
  S → aA
- It matches TOS **a** with the input symbol a.
- Then it chooses **A → b**.
- It matches TOS b with input symbol **b**.
- Both the stack and input become empty, so the string is **accepted**.

Parse Table

Non-terminal	a	b	$
S	S → aA	–	–
A	–	A → b	–

Stack	Input	Action
S$	ab$	Start
S$	ab$	Apply S → aA
aA$	ab$	Push A then a
A$	b$	Match a
A$	b$	Apply A → b
b$	b$	Push b
$	$	Match b
Empty	Empty	Accept

Parse Tree Computation

- Start with **S**.
- Apply production:

  $S \rightarrow aA$
- Apply production:

  $A \rightarrow b$
- The leaves from left to right give:

  a then b

The leaves matches the input string **ab**.

Therefore, the string **ab is accepted** by the grammar.

Parse Tree

Example2:

Given Grammar is:

S → aBa
B → bB | ε

Input String: abba$

Initial Stack: S$

Parsing Table:

Non-terminal	a	b	$
S	S → aBa	-	-
B	B → ε	B → bB	-

Predictive Parsing

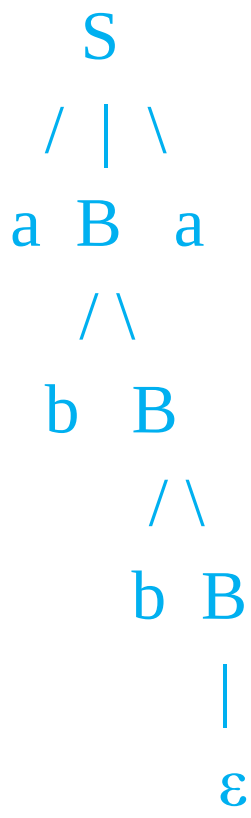
Stack	Input	Action
S$	abba$	Start
aBa$	abba$	Apply S → aBa
Ba$	bba$	Match a
bBa$	bba$	Apply B → bB
Ba$	ba$	Match b
bBa$	ba$	Apply B → bB
Ba$	a$	Match b
a$	a$	Apply B → ε
$	$	Match a
Empty	Empty	Accept

Parse Tree Construction

Given Grammar is:

S → aBa
B → bB | ε

Input String: abba



Now Using the productions selected during predictive parsing, the parse tree for the given string is as follows:

Reading leaves from left to right and ignoring ε gives string : abba

This matches with the given input string

Therefore, the given string **is accepted** by the grammar.

Example:

Given Grammar

$S \rightarrow aAa \mid BAa \mid \varepsilon$

$A \rightarrow cA \mid bA \mid \varepsilon$

$B \rightarrow b \mid \varepsilon$

Input string: bcba

	a	b	c	$
S	S → aAa	S → BAa		S → ε
A	A → ε	A → bA	A → cA	
B		B → b		

Stack	Remaining input	Action
$S	bcba$	choose S → BAa
$aAB	bcba$	choose B → b
$aAb	bcba$	match b
$aA	cba$	choose A → cA
$aAc	cba$	match c
$aA	ba$	choose A → bA
$aAb	ba$	match b
$aA	a$	choose A → ε
$a	a$	match a
$	$	Accept

Build Parse Tree Yourself

Class / Home Work

Predictive Parsing Practices

Practice 1:

$S \rightarrow aA$

$A \rightarrow bA \mid \varepsilon$

Input String: aab

Parsing Table:

Non-terminal	a	b	$
S	S → aA	—	—
A	A → bA	A → ε	A → ε

Practice 2:

$E \rightarrow T\ E'$

$E' \rightarrow +\ T\ E' \mid \varepsilon$

$T \rightarrow id$

Input String: id + id

Parsing Table:

Non-terminal	id	+	$
E	E → T E'	—	—
E'	—	E' → + T E'	E' → ε
T	T → id	—	—

LL Parsing: Introduction

- **LL Parsing** is a **top-down parsing** technique.
- The first **L** means the parser reads the input **from Left to right**.
- The second **L** means it creates a **Leftmost derivation**.

Features of LL Parsing

- It uses a **lookahead symbol** to check the next input symbol and choose the correct grammar rule.
- The parser **predicts the production rule** to use without trying different rules.
- It does **not use backtracking**, which makes parsing faster and simpler.
- LL parsing can handle many programming language grammars.

LL(k) Parser

- The value k represents the number of lookahead symbols used by the parser.
- If a parser uses k lookahead symbols, it is called an LL(k) parser.
- Example:
  - LL(1) → Uses one lookahead symbol to make a decision.

Working of LL Parser

- When the parser finds a non-terminal, it checks the lookahead symbol.
- Using this information, it selects the correct production rule.
- The selected rule is used to continue parsing the input.

LL Parser

- An LL parser is a type of predictive parser that uses a parsing table and a stack to select the correct production rule.
- It reads the input from Left to right and produces a Leftmost derivation.
- The parser uses the current non-terminal and the lookahead symbol (next input symbol) to find the correct rule from the parsing table.
- If a non-terminal has more than one production rule, the parser uses the lookahead symbol to predict the correct production.
- LL parsers do not use backtracking, so they are fast and efficient.

To build an LL parser, we need:

- The current non-terminal (top symbol to be expanded).
- The next input symbol (lookahead) to decide which production rule to apply.

Model of LL Parser

An **LL parser** has three main components:

1.Input Buffer

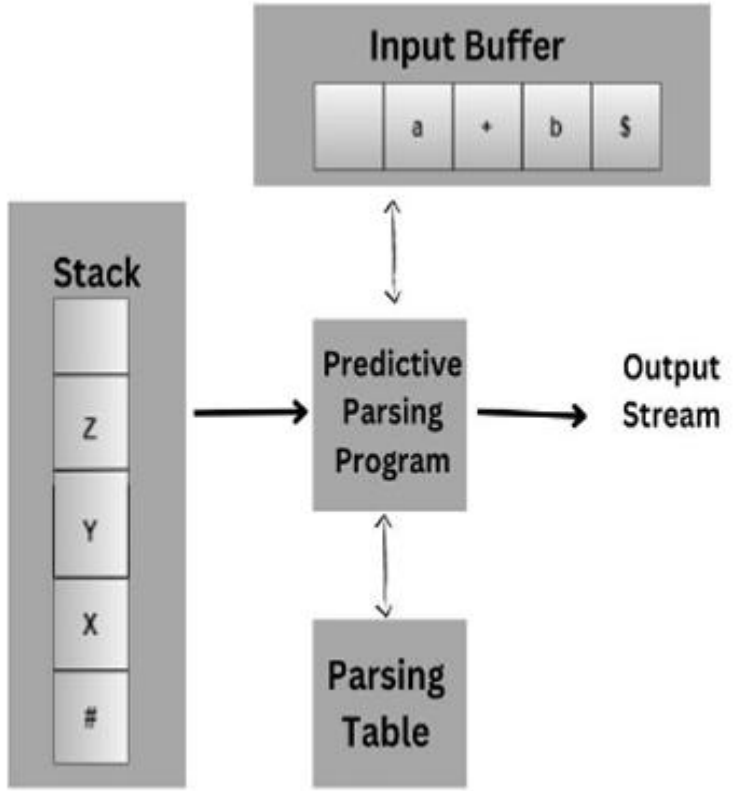
- •Stores the input string.
- •Reads input from **left to right**.
- •Uses **$** as the end marker.

2.Stack

- •Stores grammar symbols.
- •Initially contains the **start symbol** and **$**.
- •Keeps track of symbols to be processed.

3.Parsing Table

- •Selects the correct production rule.
- •Uses the **stack top** and **lookahead** symbol.
- •Built using **FIRST** and **FOLLOW** sets.



LL Parsing Process:

•If the stack top is a **terminal**, match it with the input.

•If it is a **non-terminal**, use the parsing table to choose a production rule.

Working Process of LL Parser

- Initialize the stack with the **start symbol** and **$** (end marker).
- Read the input **from left to right** using the **lookahead symbol**.
- If the top of the stack is a **terminal**, compare it with the input symbol:
  - If they match, remove both and continue.
  - If they do not match, report an error.
- If the top of the stack is a **non-terminal**, use the **parsing table** to select the correct production rule.
- Repeat the process until the **stack and input are empty**.
- If both are empty, **accept the input**; otherwise, report an error.

LL(1) Parser

- An LL(1) parser is a type of predictive parser that works without backtracking.
- It reads the input from Left to right and produces a Leftmost derivation.
- The 1 means it uses one lookahead input symbol to choose the correct production rule.
- LL(1) grammars should have no left recursion and no ambiguity.
- LL(1) parsers are simple, efficient, and easy to construct.
- Many programming language grammars are designed to be LL(1).

LL(1) Gramamar

A grammar is LL(1) if all of the following conditions are satisfied:

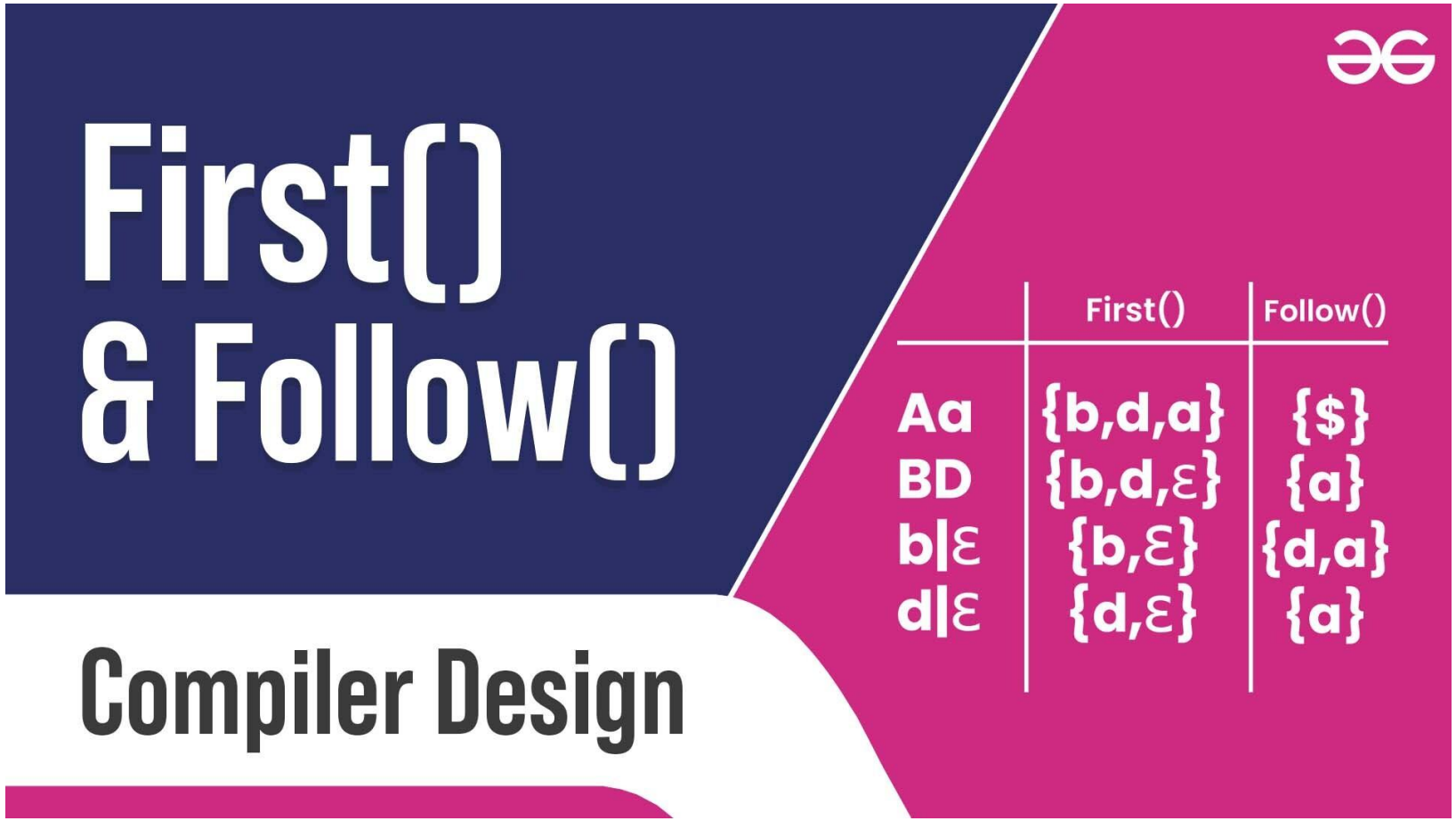
- No left recursion
- Grammar is left-factored (if needed)
- No parsing table cell contains more than one production

Requirements of LL(1) Parser

The main requirements for an **LL(1) parser** are:

- **No Left Recursion and Left Factoring**
  - The grammar should not contain left recursion and should be left-factored to avoid conflicts.
- **FIRST() and FOLLOW() Sets**
  - Used to determine the correct production rules for the parsing table.
- **Parsing Table**
  - Helps the parser select the appropriate production rule using the lookahead symbol.
- **Stack Implementation**
  - Stores grammar symbols and helps perform the parsing process.
- **Parse Tree**
  - Represents the structure of the input string according to the grammar.

FIRST and FOLLOW Computation



FIRST() Function:

- **FIRST** gives the set of terminals that can appear at the **beginning** of a string derived from a non-terminal.
- It is used to decide which production rule to choose in an LL(1) parsing table.
- First(α) is a set of terminal symbols that begin in strings derived from α

Rules for FIRST():

- ✓ Rule-1: For a production rule $X \rightarrow \in$, First(X) = { ∈ }
- ✓ Rule-2: For any terminal symbol 'a', First(a) = { a }
- ✓ Rule-3: For a production rule $X \rightarrow Y_1 | Y_2 | Y_3 \ldots . | Y_n$, then
  First(X) = First(Y_1) U First(Y_2) U First(Y_3) U…….U First(Yn)
- ✓ Rule-4: For a production rule $X \rightarrow Y_1 Y_2 Y_3 \ldots . Y_n$
  i. If **Y_1 cannot produce ε** i.e. If ∈ ∉ First(Y_1), First(X) = First(Y_1)
  ii. If **Y_1 can produce ε**: i.e. If ∈ ∈ First(Y_1), First(X) = First(Y_1) – {∈ } ∪ First($Y_2 Y_3 \ldots$)
  iii. Continue for remaining symbols:
    a. If Y_2 can also produce ε, add FIRST(Y_2) except ε and check Y_3.
    b. Continue until a symbol that **cannot produce ε** is found.
  iv. If all symbols can produce ε, add ε to First(X).

FOLLOW() Function:

- During the process of derivation, terminals that could follow the non terminal are considered as follow of that non terminal.
- **Follow()** gives the set of terminals that can appear **immediately after a non-terminal** in a grammar.
- It is used in **LL(1) parsing** to create the parsing table, especially for **ε (empty) productions**.
- **Rules for Computing Follow:**
  - ✓ Rule-1: For the start symbol S, place $ in Follow(S).
  - ✓ Rule-2: For any production rule $A \rightarrow \alpha B$, add Follow(A) to Follow(B)
  - ✓ Rule-3: For any production rule $A \rightarrow \alpha B\beta$,
    a. If $\in \notin$ First(β), then add First(β) to Follow(B)
    b. If $\in \in$ First(β), then add { First(β) – $\in$ } and Follow(A) to Follow(B)

Importance:

- FOLLOW sets help decide where to place **ε-productions** in the LL(1) parsing table.
- They help the parser know what symbol can come after a non-terminal.

Example 1:

Given Grammar:

$S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC / \in$

$D \rightarrow EF$

$E \rightarrow g / \in$

$F \rightarrow f / \in$

Compute first() and Follow()

Simple Rule to Remember to compute

FIRST(XY)

- Take FIRST of the first symbol **X**.
- If X can produce **ε**, then also consider FIRST of the next symbol **Y**.
- Continue until a symbol cannot produce ε.

Computation of First():

We know that

First(terminals)= {terminal} Rule 2

First(ε) = {ε} Rule 1

Final First()

FIRST(S) = {a}
FIRST(B) = {c}
FIRST(C) = {b, ε}
FIRST(D) = {g, f, ε}
FIRST(E) = {g, ε}
FIRST(F) = {f, ε}

- FIRST(E) = FIRST(g) ∪ FIRST(ε)
  = {g} ∪ {ε}
  = {g, ε} Rule-3
- FIRST(F) = FIRST(f) ∪ FIRST(ε)
  = {f} ∪ {ε}
  = {f, ε} Rule-3
- FIRST(C) = FIRST(bC) ∪ FIRST(ε)
  = {b} ∪ {ε} Rule-4
  = {b, ε} Rule-3
- FIRST(B) = FIRST(c)
  = {c} Rule-4
- FIRST(D) = FIRST(E) − {ε} ∪ FIRST(F)
  = {g} ∪ {f, ε}
  = {g, f, ε} Rule-4:
- FIRST(S) = FIRST(a)
  = {a} Rule-4

Computation of Follow():

- We already have First()
- For the start symbol S,

  Follow(S) = { $ } Rule-01
- **Production S → aBDh** Rule 03
- For Follow(B),

  β = **Dh** and First(Dh) = {g, f, h}

Since ε is **not** in First(Dh),

  Follow(B) = {g, f, h}

For Follow(**D**), β = **h** and First(h) = {h}

  Follow(D) = {h}

- **Production D → EF** Rule 03
- For Follow(E), β = **F** and First(F) = {f, ε}

  Since ε ∈ First(F)

Follow(E) = {First(F) − ε} ∪ Follow(D)
= {f} ∪ {h}
= {f, h}

Production D → EF Rule 02

For Follow(F)

Follow(F) = Follow(D) = {h}

Production B → cC Rule 02

For Follow(C)

Follow(C) = Follow(B)
= {g, f, h}

Final Follow()

Follow(S) = { $ }
Follow(B) = {g, f, h}
Follow(D) = {h}
Follow(E) = {f, h}
Follow(F) = { h }
Follow(C) = {g, f, h}

Example 2:

S → AB

A → a | ε

B → bB | c

FIRST Computation:

FIRST(A):

A → a → add **a**

A → ε → add **ε**

So, FIRST(A) = { a, ε }

FIRST(B):

B → bB → add **b**

B → c → add **c**

So, FIRST(B) = { b, c }

FIRST(S):

In S → AB

FIRST(A) contains **a**, so add **a**

A can produce ε, so check B

FIRST(B) contains **b, c**, so add them

So, **FIRST(S) = { a, b, c }**

Final FIRST()

FIRST(S) = { a, b, c }

FIRST(A) = { a, ε }

FIRST(B) = { b, c }

FOLLOW Computation

Since S is the start symbol:

FOLLOW(S) = { $ }

FOLLOW(A):

From S → AB, As A is followed by B

Add FIRST(B) − {ε} to FOLLOW(A)

FIRST(B) = { b, c }

Thus FOLLOW(A) = { b, c }

FOLLOW(B): From S → AB

As B is at the end, so add FOLLOW(S) to Follow(B)

FOLLOW(B) = { $ }

Final FOLLOW()

FOLLOW(S) = { $ }

FOLLOW(A) = { b, c }

FOLLOW(B) = { $ }

Example 3:

S → A

A → aBA'

A' → dA' | ε

B → b

C → g

Compute first() and Follow()

FIRST Computation

FIRST(S) = { a }

FIRST(A) = { a }

FIRST(A’) = { d, ε }

FIRST(B) = { b }

FIRST(C) = { g }

FOLLOW Computation

FOLLOW(S) = { $ }

FOLLOW(A) = { $ }

FOLLOW(A’) = { $ }

FOLLOW(B) = { d, $ }

FOLLOW(C) = { }

Compute the FIRST and FOLLOW of all the non-terminals

Practice 1: TU- 2081

$S \rightarrow AB$
$A \rightarrow 0A' \mid 1A' \mid \varepsilon$
$A' \rightarrow SSA' \mid \varepsilon$
$B \rightarrow AS \mid 1$

Practice 2: TU- 2080

$S \rightarrow aAbCD \mid \varepsilon$

$A \rightarrow SD \mid \varepsilon$

$C \rightarrow Sa$

$D \rightarrow aBD \mid \varepsilon$

Practice 3:

$S \rightarrow (L) \mid a$

$L \rightarrow SL'$

$L' \rightarrow ,SL' \mid \varepsilon$

Practice 4:

$S \rightarrow ACB \mid CbB \mid Ba$

$A \rightarrow da \mid BC$

$B \rightarrow g \mid \varepsilon$

$C \rightarrow h \mid \varepsilon$

Construction of LL(1) Parsing Table

- An **LL(1) parsing table** is built using the **FIRST** and **FOLLOW** sets.
- It helps the parser to select the correct production rule using the **non-terminal** and **lookahead symbol**.

Construct LL(1) Parsing Table : Steps

- Create a table
- Rows represent non-terminals and Columns represent terminals and the end symbol $
- For each production rule: A → α in grammar
  - Find FIRST(α).
  - For every terminal **a** in FIRST(α)
    - Add A → α in table entry M[A, a]
- If ε is in FIRST(α),
  - then for every terminal **a** in FOLLOW(A), add A → α in M[A, a]
- If ε is in FIRST(α) and $ is in FOLLOW(A),
  - Add A → α to M[A, $]
- If any table entry contains more than one production rule, the grammar is not LL(1).

Practice 1: Construct LL(1) Parser

S → aABb

A → c | ε

B → d | ε

Find FIRST()

FIRST(A)={c,ϵ}

FIRST(B)={d,ϵ}

FIRST(S)={a}

Find FOLLOW()

FOLLOW(S)={$}

FOLLOW(A)={ d, b}

FOLLOW(B)={b}

Now Construct LL(1) Parse Table

- For S → aABb, FIRST(aABb)={a},

  Add M[S, a] = S → aABb

- For A → c, FIRST(c)={c}

  Add M[A, c] = A → c

- For A → ε

  Since ε ∈ FIRST(ε), Use FOLLOW(A)={ d, b}

  Add M[A, d] = A → ε and M[A, b] = A → ε

- For B → d

  FIRST(d)={d}, Add M[B, d] = B → d

- For B → ε

  Since ε ∈ FIRST(ε), Use FOLLOW(B)={b}

  Add M[B, b] = B → ε

Non-Terminal	a	b	c	d	$
S	S → aABb				
A		A → ε	A → c	A → ε	
B		B → ε		B → d	

Final LL(1) Parse Table

Parsing of input string

w = acdb

We have

Non-Terminal	a	b	c	d	$
S	S → aABb				
A		A → ε	A → c	A → ε	
B		B → ε		B → d	

Predictive Parsing Process:

Stack	**Input**	**Action**
S$	acdb$	Use S → aABb
aABb$	acdb$	Match a
ABb$	cdb$	Use A → c
cb$	cdb$	Match c
Bb$	db$	Use B → d
db$	db$	Match d
b$	b$	Match b
$	$	Accept

Class Work:

Show parsing of string w = ab , acb

Example 2:

E → T E'

E' → + T E' | ε

T → id

Find FIRST Sets

FIRST(T) = { id }

FIRST(E') = { +, ε }

FIRST(E) = { id }

Find FOLLOW Sets

FOLLOW(E) = { $ }

FOLLOW(E') = { $ }

FOLLOW(T) = { +, $ }

Constructing LL(1) Parsing Table

For E → T E‘ FIRST(TE'): { id }

Add M[E, id] = E → T E‘

For E' → + T E‘ FIRST(+TE'): { + }

Add M[E', +] = E' → + T E‘

For E' → ε

Since ε ∈ FIRST(ε), Use FOLLOW(E') = { $ }

Add M[E', $] = E' → ε

T → id FIRST(id): { id }

Add M[T, id] = T → id

Class Work:
Show parsing of string
w = id + id , id + id + id

Use Parse table.

Non-Terminal	id	+	$
E	E → T E'		
E'		E' → + T E'	E' → ε
T	T → id		

Final LL(1) Parsing Table

Example 3:

Construct LL(1) Parse Table

E → T E'

E' → + T E' | ε

T → F T'

T' → * F T' | ε

F → id | (E)

First() and Follow() computation

Symbol	FIRST	FOLLOW
E	{ id, (}	{ $,) }
E'	{ +, ε }	{ $,) }
T	{ id, (}	{ +, $,) }
T'	{ *, ε }	{ +, $,) }
F	{ id, (}	{ *, +, $,) }

Non-Terminal	id	+	*	(	)	$
E	E → TE'			E → TE'		
E'		E' → +TE'		—	E' → ε	E' → ε
T	T → FT'			T → FT'		
T'		T' → ε	T' → *FT'	—	T' → ε	T' → ε
F	F → id			F → (E)		

Final LL(1) Parse Table

Class Work:

Show parsing of string w = id + id, id * id, (id), id + id* id

Use Parse table.

Construct LL(1) parsing table for following grammar.

Also check they are LL(1) grammar or not?

Practice 1:

S → AB

A → aA | ε

B → bB | ε

Practice 2:

S → A | a

A → a

Practice 3:

E → E + T | T

T → T * F | F

F → (E) | id

Practice 4:

S->aA/ bAc

A-> c/ ε

Practice 5:

S → aA

S → aB

A → c

B → d

LL(1) Grammars and Languages

LL(1) Grammar

- A context-free grammar (CFG) is called an LL(1) grammar if its LL(1) parsing table has no multiple entries (no conflicts).
- This means the parser can choose only one production rule for every non-terminal and lookahead symbol.

Meaning of LL(1)

- First L (Left-to-right): The parser reads the input from left to right.
- Second L (Leftmost derivation): The parser constructs a leftmost derivation.
- 1: The parser uses one lookahead input symbol to select the correct production rule.

LL(1) Language

- A language is called an **LL(1) language** if it can be generated by an **LL(1) grammar**.

Properties of LL(1) Grammar

An LL(1) grammar:

- Has **no ambiguity** i.e. every string has only one parse tree.
- Has **no left recursion**.
- Has **no conflicts** in the LL(1) parsing table i.e.each table entry contains at most one production.
- Can be parsed **without backtracking**.
- Uses **one lookahead symbol** to predict the correct production rule.

Advantages of LL Parsing

- Simple to understand and implement.
- Fast and efficient because it does not use backtracking.
- Uses predictive parsing, which helps detect and report errors easily.
- Suitable for many programming language grammars.
- Can be automatically generated for LL(1) grammars.

Disadvantages of LL Parsing

- Works only for LL(1) grammars (or LL(k) grammars with *k* lookahead symbols).
- Cannot handle left-recursive grammars.
- Left factoring may be required before constructing the parser.
- If lookahead symbols (k) are is large, the parser becomes more complex and slower.
- Cannot parse ambiguous grammars.